

Why Odoo Was Our Inspiration

Our group's starting reference point was Odoo, a real-world enterprise transaction system used for sales, inventory, and order management. We looked at Odoo specifically because our assignment required us to study a transaction system that applies a sortable feature to both digits and text.

What we found in Odoo:

- Odoo's Kanban view (used in Sales and CRM) lets users drag transaction cards to reorder or move them between pipeline stages (e.g., New → Sent → Validated). This is a native drag-and-drop sortable behavior on transaction records.
- Odoo's List view lets users click column headers to sort transaction tables by either text fields (customer name) or numeric fields (amount, date), covering the “digits and text” requirement from a different angle.

What we took from Odoo into our own system:

- Transaction records should be sortable as whole units, not just individual fields — position in a list can represent priority, not only alphabetical or numeric order.
- A transaction naturally mixes text (names, item descriptions) and digits (amounts, quantities, IDs, timestamps), and a good sortable system needs to handle both cleanly.

Where we scaled it down: Odoo is a full enterprise ERP spanning many modules. For this project, we scoped the concept down into one concrete, buildable workflow — a restaurant counter-to-kitchen order queue — built as a single system with two panels, described below.

System Architecture — One File, Two Panels

The system is one HTML file with two panels sharing a single JavaScript state. It is not two separate files.

Why: without a database, two truly separate files would need an extra sync mechanism (e.g., localStorage cross-tab events) just to talk to each other. That adds risk without adding anything the assignment is actually testing. One file guarantees both panels always work together, live, with no sync step to break during a demo.

Panel	Contains
Left panel Order Pad (customer / front-of-house)	Table selector, menu list with quantity steppers, running total, Send to Kitchen button.
Right panel Kitchen Queue (back-of-house)	Sort-mode buttons (Priority / Amount / Table / Longest waiting), draggable ticket rail, Mark Ready button, Completed list.

Flow: an order created in the left panel appears immediately in the right panel

Requirements Checklist (Priority Order)

Ordered by priority. Items 1–2 are the actual assignment requirement and must work reliably. Items 3–4 make it a believable transaction system. Item 5 is polish.

Priority 1 — Sortable (core requirement)

- Drag-and-drop reordering of kitchen tickets using jQuery UI sortable(), with a visible drag handle.
- Dragging updates the underlying order data, not just the on-screen position.
- Sort-by-field toggle: amount (digits), table name (text), and longest-waiting (time), in addition to manual drag priority.

Priority 2 — Core transaction flow

- Order pad: select a table, select item(s) and quantity, see a running total, submit the order.
- Submitted order appears in the kitchen queue immediately, with no page reload.
- Kitchen can mark an order ready, removing it from the active queue.

Priority 3 — Supporting features

- Completed / order history list, separate from the active queue.
- Basic validation: cannot submit an empty order.
- Empty-state message when the queue has no active orders.

Priority 4 — Polish

- Aging indicator on tickets that have been waiting a while.
- Entry animation for newly submitted tickets.

Feature Walkthrough

How each feature works, for the “how jQuery UI was used” part of the paper.

Sort-by-field toggle (digits and text)

Four buttons (Priority / Amount / Table / Longest waiting) change a sortMode variable. When sortMode is not "manual", a sorted copy of the orders array is built with a different comparator per mode:

- Amount (digits): sorts numerically, highest first.
- Table (text): sorts alphabetically using localeCompare().
- Longest waiting (time): sorts by creation timestamp, oldest first.

Dragging is disabled while a non-manual sort is active. Switching back to “Priority” restores dragging and the manual order, preserved untouched in the background.

Completed section

“Mark Ready” removes the order from the orders array (splice) and pushes it into a separate completed array with a timestamp. The two arrays render into two different parts of the page, which is why finished orders disappear from the active rail instead of staying there greyed out.

Aging color and “next up” tag

- Wait time = difference between now and the order's createdAt timestamp, in minutes.
- 3+ minutes = warning color; 6+ minutes = urgent color. The queue re-renders every 20 seconds so this updates on its own.

Task Assignment

Member	Role	Task	Deliverable
Thea	Paper / Documentation	Write the full paper: Odoo research summary, system explanation, design rationale, and how jQuery UI sortable() was applied. Consolidate everyone's input into one final document.	Final paper (Word)
Nick	Research — Odoo / Reference System	Researched Odoo's Kanban and List view sortable behavior in more depth (screenshots, short explanations). Identify 1–2 more reference systems with sortable transaction features for comparison.	Research notes + screenshots for the paper
Nick, Daniela, Thea	Order Pad (left panel)	Build/refine the customer-facing order pad: table selector, menu selection, quantity steppers, total calculation, sending orders into the queue.	Working order pad panel
Mcnesse, Barro, Thea	Kitchen Queue & Sortable Logic (right panel)	Build/refine the kitchen ticket rail: jQuery UI sortable() drag behavior, the sort-by-field toggle (amount / table / wait time), and the completed-orders section.	Working kitchen queue panel
Thea	Testing, Polish & Presentation	Test the system end-to-end (edge cases: empty queue, dragging while sorted, etc.), suggest/apply small UI polish, and prepare the demo flow for presentation day.	Bug/QA notes + presentation outline